

MIT 6.S191 – Introduction to Deep Learning

Lecture 4: Deep Generative Modeling Detailed Study Notes

Personal study notes

Lecture: January 6, 2026

Contents

1	What is Generative Modeling?	2
1.1	Why generative models matter	2
2	Latent Variables: Hidden Explanatory Factors	4
3	Autoencoders	4
3.1	Encoder, bottleneck, and decoder	4
3.2	Why the bottleneck matters	6
3.3	Why a traditional autoencoder is not yet a good generative model	7
4	Variational Autoencoders (VAEs)	7
4.1	From a point to a distribution	7
4.2	The two-part VAE objective	8
4.3	Why regularize the latent space? Continuity and completeness	9
5	The Reparameterization Trick	10
5.1	The differentiability problem	10
5.2	Move the randomness into an external variable	10
6	Interpreting and Traversing a VAE Latent Space	11
7	Generative Adversarial Networks (GANs)	11
7.1	Generator and discriminator	11
7.2	Intuition: an adversarial game	12
8	GAN Objective Function	13

8.1 Why GAN training can be difficult	14
9 Generating New Data with GANs	15
10 GANs as Distribution Transformers	15
11 CycleGAN: Unpaired Domain Translation	15
12 Autoencoders, VAEs, and GANs: Conceptual Comparison	17
13 Lecture 4 Big Picture	17
14 Quick Review Questions	18

1 What is Generative Modeling?

The central idea of **generative modeling** is to learn the probability distribution that produced the observed data. Rather than only learning a mapping from an input x to a label y , a generative model tries to capture the structure of the data distribution itself. Once a useful approximation to that distribution has been learned, the model can reason about likely and unlikely observations and can also **sample** from the learned distribution to create new data instances.

In supervised learning, the training set contains pairs (x, y) and the objective is typically to learn a function f such that $f(x) \approx y$. Classification, regression, object detection, and semantic segmentation are examples. In the unsupervised setting emphasized in this lecture, we may only observe x . The learning problem becomes: *what hidden structure explains the observations?*

Generative modeling

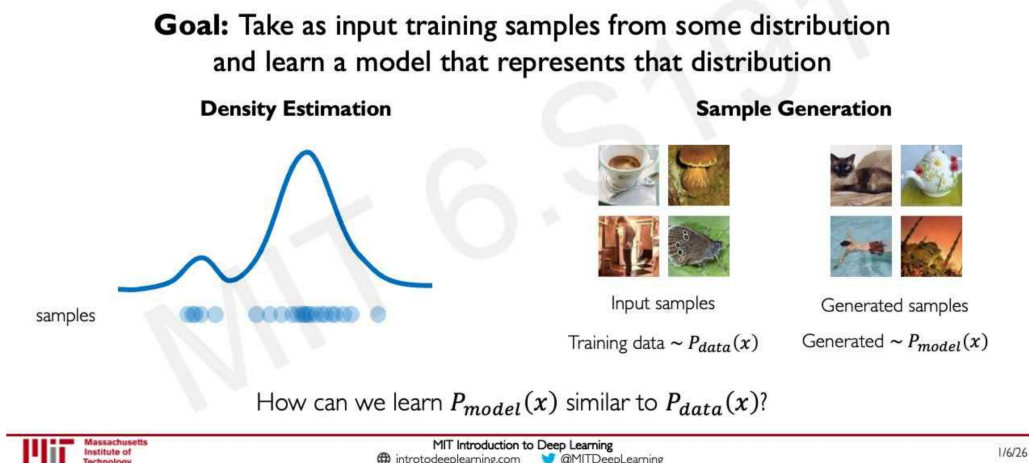


Figure 1: The lecture frames generative modeling as learning a model distribution from observed training samples, enabling density estimation and sample generation. (MIT 6.S191 Lecture 4 slide 5).

Key idea

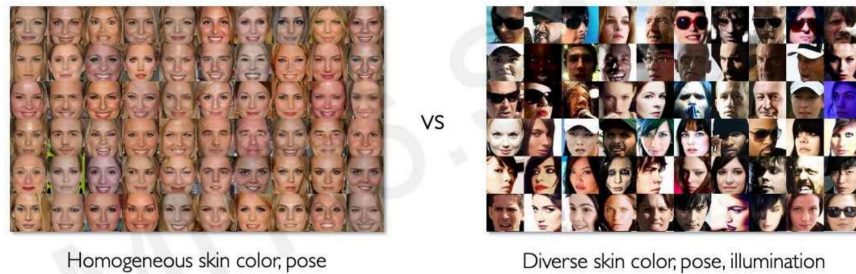
If the true data follow $p_{data}(x)$, the model tries to learn $p_{model}(x)$ so that the model distribution is close to the data distribution. Two important consequences are **density estimation** (describing where data are likely to occur) and **sample generation** (drawing new x values that resemble the training distribution).

1.1 Why generative models matter

The lecture emphasizes that generation is not limited to photorealistic images. Learning a data distribution can reveal underlying factors of variation, identify rare or unusual observations, and support generation in many modalities.

Why generative models? Debiasing

Capable of uncovering **underlying features** in a dataset



How can we use this information to create fair and representative datasets?



Software Lab!



Massachusetts
Institute of
Technology

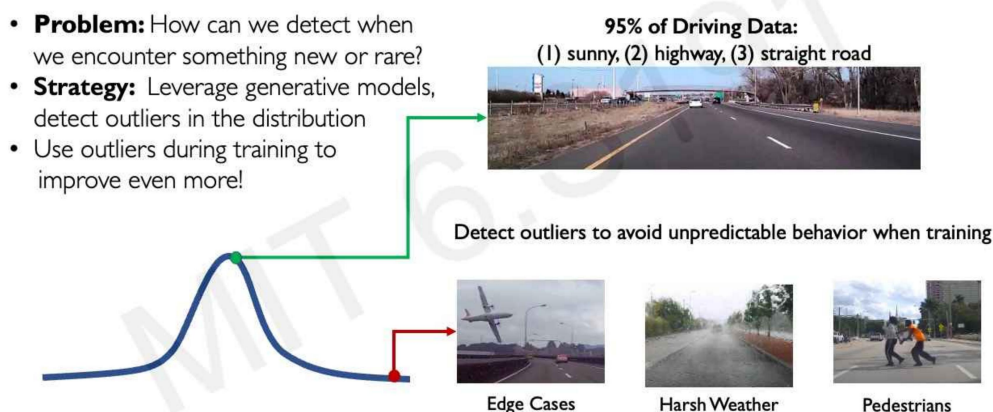
MIT Introduction to Deep Learning
introtodeeplearning.com @MITDeepLearning

Amini/Soleimany+ AAAI/AIES 2019. 1/6/26

Figure 2: Generative models can uncover underlying features of a dataset, which can be useful when studying representation and dataset bias. (MIT 6.S191 Lecture 4 slide 6).

Debiasing and representation. If a training set is concentrated around only a narrow subset of the real population, a learned model can inherit that imbalance. Modeling the underlying distribution provides a way to inspect which factors are represented and which are underrepresented. In the lecture's face example, this motivates learning latent features that can be used to create a more representative training strategy.

Why generative models? Outlier detection



Massachusetts
Institute of
Technology

MIT Introduction to Deep Learning
introtodeeplearning.com @MITDeepLearning

Amini+ /ROS 2018. 1/6/26

Figure 3: Outlier detection: observations far from the high-density region can correspond to rare or unfamiliar situations. (MIT 6.S191 Lecture 4 slide 7).

Outlier detection. If a model has learned what “typical” data look like, low-probability observations can be treated as potential outliers. This is especially relevant in domains such as autonomous driving and medicine, where uncommon cases may be disproportionately important.

Sample generation. Once the distribution has been learned, new examples can be sampled from it. This principle underlies generative systems for language, images, video, molecules, and other structured data.

2 Latent Variables: Hidden Explanatory Factors

A **latent variable** is an unobserved variable that helps explain variation in the observed data. The lecture uses Plato’s “Myth of the Cave” as an analogy: the prisoners directly observe shadows, not the objects producing them. Likewise, we directly observe data x , but there may be hidden factors z that explain why the observations differ.

What is a latent variable?



Can we learn the **true explanatory factors**, e.g. latent variables, from only observed data?

Figure 4: The cave analogy motivates the question: can hidden explanatory factors be inferred from observed data alone? (MIT 6.S191 Lecture 4 slide 11).

A useful latent representation should compress the important axes of variation in the data. For faces, latent dimensions might capture properties such as pose, illumination, expression, or other features. Importantly, the neural network is not necessarily given names for these factors; it learns a representation through its training objective.

3 Autoencoders

3.1 Encoder, bottleneck, and decoder

An **autoencoder** learns to reconstruct its own input. It consists of two mappings:

$$\text{Encoder: } z = f_{\phi}(x), \quad \text{Decoder: } \hat{x} = g_{\theta}(z).$$

The encoder maps a high-dimensional observation x into a lower-dimensional latent representation z . The decoder maps z back into the original data space to produce a reconstruction $\hat{x}$.

Autoencoders: background

How can we learn this latent space?
Train the model to use these features to **reconstruct the original data**

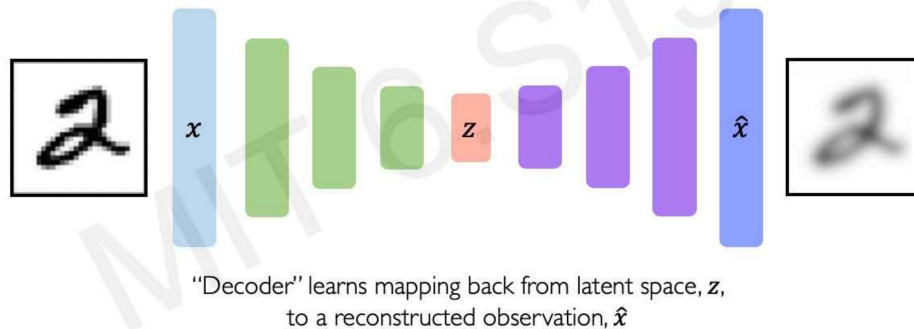


Figure 5: The encoder learns a mapping from the original observation x to a lower-dimensional latent representation z . (MIT 6.S191 Lecture 4 slide 14).

Autoencoders: background

How can we learn this latent space?
Train the model to use these features to **reconstruct the original data**

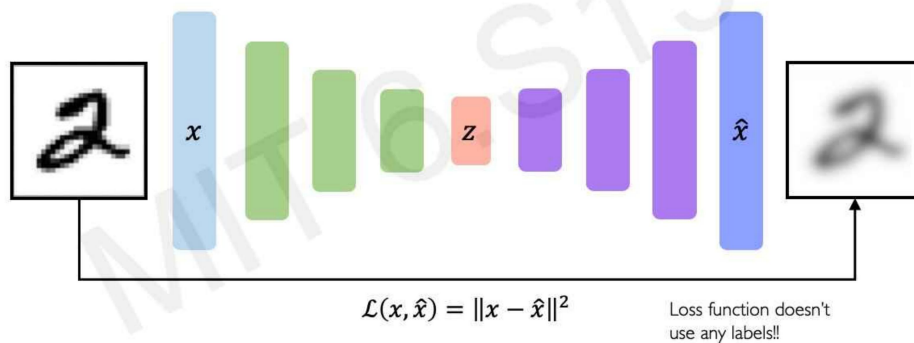


Figure 6: The decoder maps the latent representation back to a reconstruction $\hat{x}$; reconstruction itself supplies the training signal. (MIT 6.S191 Lecture 4 slide 15).

No class label y is required. The target is simply the original input. A common reconstruction loss for continuous-valued data is mean-squared error:

$$\mathcal{L}_{\text{recon}}(x, \hat{x}) = \|x - \hat{x}\|_2^2.$$

Depending on the assumed output distribution, other reconstruction objectives can also be used. The lecture's central point is that the loss depends on the relationship between x and $\hat{x}$, not on external labels.

3.2 Why the bottleneck matters

The latent vector is an information bottleneck. If z has fewer dimensions than x , the network cannot simply preserve every input coordinate independently; it is pressured to encode the information most useful for reconstruction.

Dimensionality of latent space $\rightarrow$ reconstruction quality

Autoencoding is a form of compression!
Smaller latent space will force a larger training bottleneck

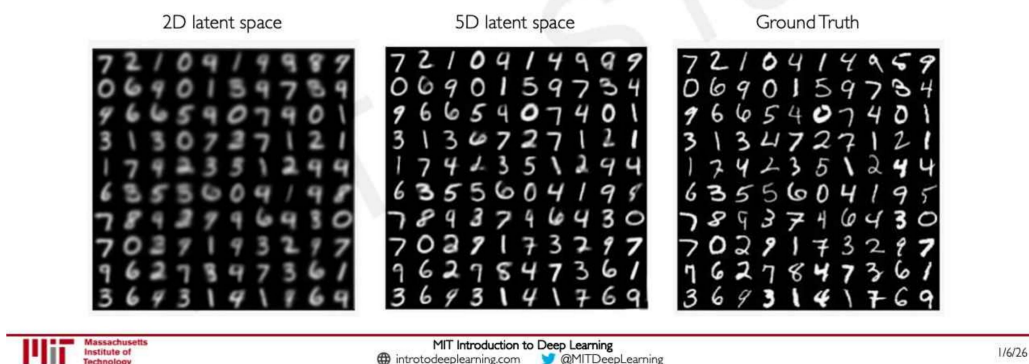


Figure 7: Smaller latent spaces create stronger compression bottlenecks and generally make exact reconstruction more difficult. (MIT 6.S191 Lecture 4 slide 17).

A very small latent space may discard important information, causing poor reconstructions. A larger latent space can preserve more information, improving reconstruction, but if the bottleneck is too weak the model may learn a less useful compressed representation. Thus latent dimensionality controls a tradeoff between **compression** and **reconstruction fidelity**.

Autoencoders for representation learning

Bottleneck hidden layer forces network to learn a compressed latent representation

Reconstruction loss forces the latent representation to capture (or encode) as much “information” about the data as possible

Autoencoding = **Automatically encoding** data; “Auto” = **self**-encoding

Figure 8: Autoencoding as representation learning: the bottleneck encourages compression while reconstruction loss pressures the representation to retain useful information. (MIT 6.S191 Lecture 4 slide 18).

Intuition

The encoder cannot be trained merely by saying “learn useful features,” because no labels define what useful means. Reconstruction provides a self-supervised signal: a latent code is useful if the decoder can use it to recover the original observation.

3.3 Why a traditional autoencoder is not yet a good generative model

A standard autoencoder is deterministic: for a fixed trained network and a fixed input x , the encoder produces the same z and the decoder produces the same $\hat{x}$. More importantly, the latent space is not explicitly forced to follow a known probability distribution. Arbitrary points in latent space may therefore decode to meaningless outputs. This motivates the probabilistic extension introduced next.

4 Variational Autoencoders (VAEs)

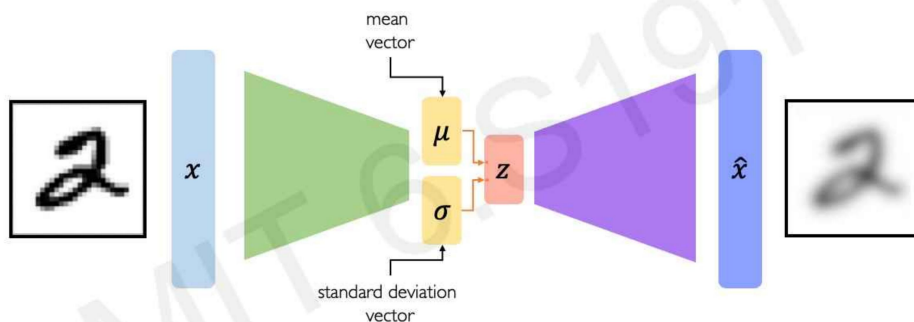
4.1 From a point to a distribution

A **variational autoencoder** adds a probabilistic interpretation to the latent representation. Instead of encoding an input to one fixed point z , the encoder predicts parameters of a distribution over z . In the common Gaussian formulation, the encoder produces a mean vector $\mu(x)$ and standard-deviation vector $\sigma(x)$:

$$q_{\phi}(z | x) = \mathcal{N}(z; \mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x))).$$

A latent sample is drawn from this distribution and passed to the decoder.

VAEs: key difference with traditional autoencoder



Variational autoencoders are a probabilistic twist on autoencoders!

Sample from the mean and standard deviation to compute latent sample

Figure 9: Key difference from a traditional autoencoder: the VAE encoder predicts distribution parameters μ and σ , then samples z . (MIT 6.S191 Lecture 4 slide 22).

The encoder is therefore an **inference model** $q_{\phi}(z | x)$, while the decoder defines $p_{\theta}(x | z)$, a distribution over observations conditioned on latent variables. The parameters ϕ and θ are learned jointly.

4.2 The two-part VAE objective

The VAE loss has two competing goals:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{reconstruction}} + \mathcal{L}_{\text{regularization}}.$$

The reconstruction term encourages z to preserve information needed to reproduce x . The regularization term encourages the learned latent distribution to remain close to a chosen prior $p(z)$.

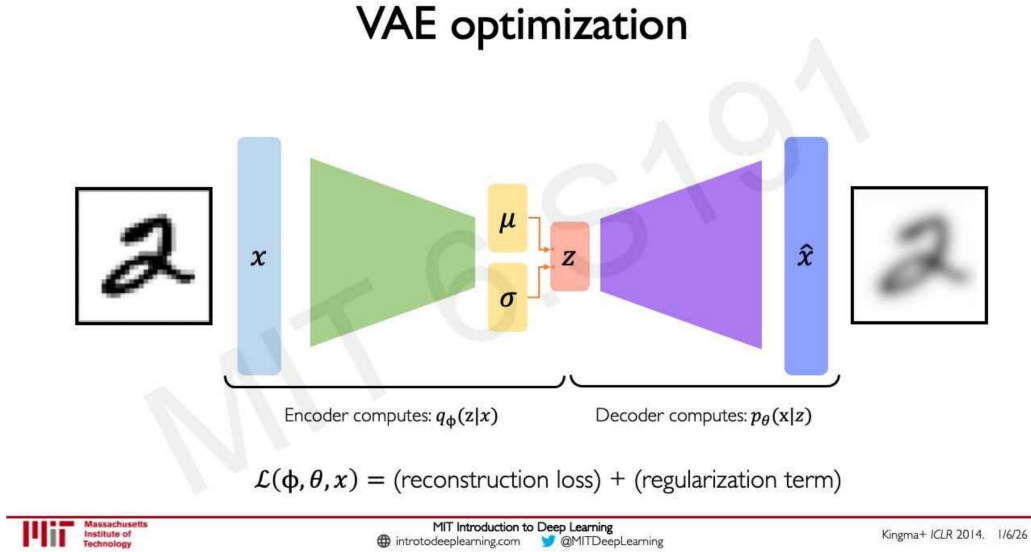


Figure 10: VAE optimization combines reconstruction and distributional regularization while training encoder and decoder together. (MIT 6.S191 Lecture 4 slide 24).

A standard choice is

$$p(z) = \mathcal{N}(0, I),$$

and the distributional distance is commonly the Kullback–Leibler divergence:

$$D_{\text{KL}}(q_{\phi}(z | x) \| p(z)).$$

For a diagonal Gaussian posterior and a unit Gaussian prior, this term has a closed form. Using variance σ_j^2 for latent dimension j ,

$$D_{\text{KL}}(q \| p) = \frac{1}{2} \sum_j \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right).$$

Thus a common minimization objective is

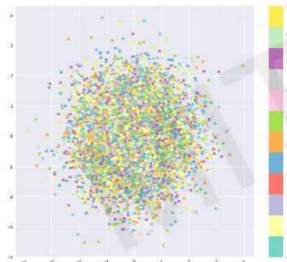
$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + D_{\text{KL}}(q_{\phi}(z | x) \| p(z)).$$

Priors on the latent distribution

$$D(q_{\phi}(z|x) \parallel p(z))$$

$$= -\frac{1}{2} \sum_{j=0}^{k-1} (\sigma_j + \mu_j^2 - 1 - \log \sigma_j)$$

KL-divergence
between the two
distributions



Common choice of prior – Normal Gaussian:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

- Encourages encodings to distribute evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (i.e., by memorizing the data)

Figure 11: The normal prior regularizes the inferred latent distribution; the lecture expresses the distributional penalty using KL divergence. (MIT 6.S191 Lecture 4 slide 29).

4.3 Why regularize the latent space? Continuity and completeness

The lecture highlights two desirable properties.

Continuity: nearby points in latent space should decode to observations that are similar in meaning/content. If two nearby z values decode to completely unrelated examples, smoothly moving through latent space is not meaningful.

Completeness: points sampled from relevant regions of latent space should decode to meaningful observations. We do not want large “holes” where a sampled latent vector produces nonsense.

Intuition on regularization and the Normal prior

What properties do we want to achieve from regularization? 🤔

1. **Continuity:** points that are close in latent space → similar content after decoding
2. **Completeness:** sampling from latent space → “meaningful” content after decoding

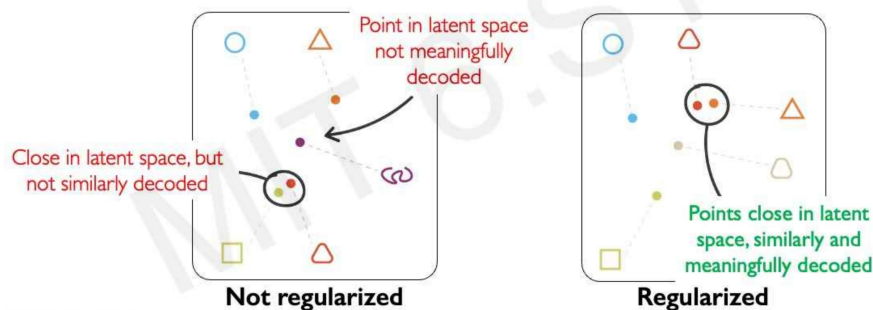


Figure 12: Regularization is motivated by continuity and completeness: nearby latent points should decode similarly, and sampled regions should remain meaningful. (MIT 6.S191 Lecture 4 slide 30).

The normal prior helps by encouraging latent encodings to occupy a coherent, centered region rather than forming arbitrary isolated clusters. Reconstruction prevents all encodings from collapsing to the same point: the model must still retain enough information to reconstruct different inputs. VAE training is therefore a balance between **faithful reconstruction** and a **well-structured latent distribution**.

5 The Reparameterization Trick

5.1 The differentiability problem

Training neural networks requires gradients. Directly writing

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

places a stochastic sampling operation in the path between the encoder parameters and the loss. The lecture presents this as the key obstacle: we need randomness for a probabilistic latent representation, but we also need a computation graph through which gradients can be propagated.

5.2 Move the randomness into an external variable

The solution is the **reparameterization trick**. Draw

$$\epsilon \sim \mathcal{N}(0, I)$$

and construct

$$z = \mu + \sigma \odot \epsilon.$$

Now the random component is isolated in ϵ , while z is a differentiable function of the encoder outputs μ and σ for a fixed draw of ϵ .

Reparametrizing the sampling layer

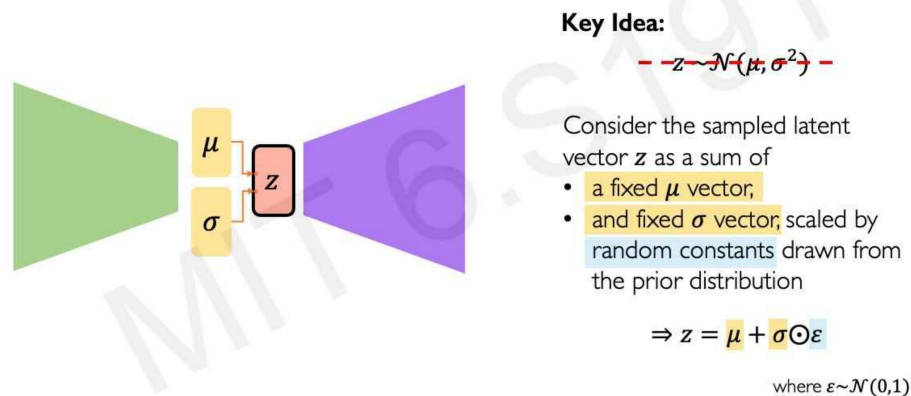


Figure 13: Reparameterization expresses the sampled latent vector as $z = \mu + \sigma \odot \epsilon$, with ϵ drawn from a fixed prior. (MIT 6.S191 Lecture 4 slide 35).

This matters because gradients can flow from the decoder and loss through z into both μ and σ , and therefore into the encoder weights ϕ . The network remains stochastic during training, but the learnable path is differentiable.

Key idea

The reparameterization trick does *not* remove randomness. It relocates randomness into ϵ , allowing the model to differentiate with respect to the parameters that control the distribution.

6 Interpreting and Traversing a VAE Latent Space

A well-trained VAE can organize latent dimensions so that changing a coordinate changes some property of the decoded output. The lecture demonstrates **latent perturbation**: start with a latent code, slightly increase or decrease one latent coordinate, decode again, and observe how the output changes.

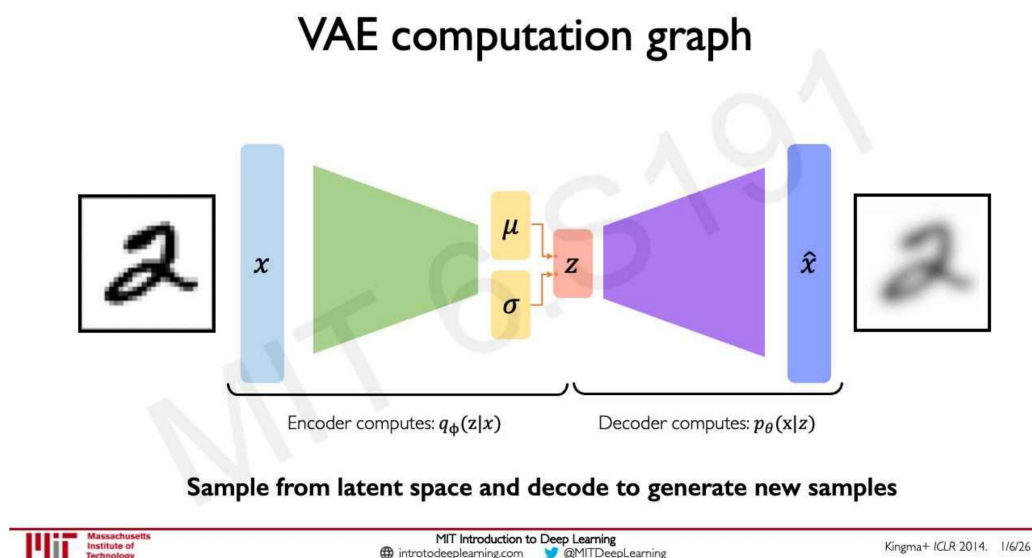


Figure 14: Latent perturbation illustrates how changes in latent coordinates can correspond to systematic changes in reconstructed examples. (MIT 6.S191 Lecture 4 slide 38).

This provides a degree of control and interpretability. More generally, interpolating between latent points can produce gradual changes in decoded samples when the space has learned the desired continuity.

7 Generative Adversarial Networks (GANs)

VAEs emphasize learning a structured latent representation together with reconstruction. GANs take a different route: if the primary goal is high-quality sample generation, can a network directly transform simple random noise into samples that follow the complex data distribution?

7.1 Generator and discriminator

A GAN contains two networks:

- **Generator G :** maps noise $z \sim p(z)$ to a generated sample $G(z)$. Its goal is to create samples that resemble real data.
- **Discriminator D :** receives either real data x or generated data $G(z)$ and outputs a score/probability indicating whether the input is real.

VAE summary

- I. Compress representation of world to something we can use to learn

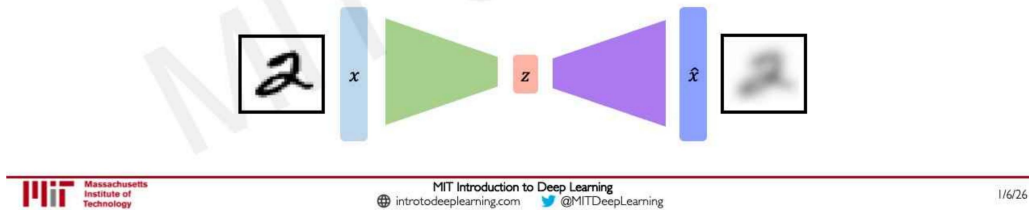


Figure 15: A GAN pits a generator against a discriminator: the generator synthesizes examples from noise while the discriminator tries to distinguish generated examples from real data. (MIT 6.S191 Lecture 4 slide 43).

7.2 Intuition: an adversarial game

Initially, the generator produces poor samples because its weights are random. The discriminator can learn to separate real and generated examples. The generator then receives a learning signal that pushes its outputs toward regions that the discriminator considers real. As the generator improves, the discriminator must also improve. This alternating competition drives the generated distribution toward the real data distribution.

VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)

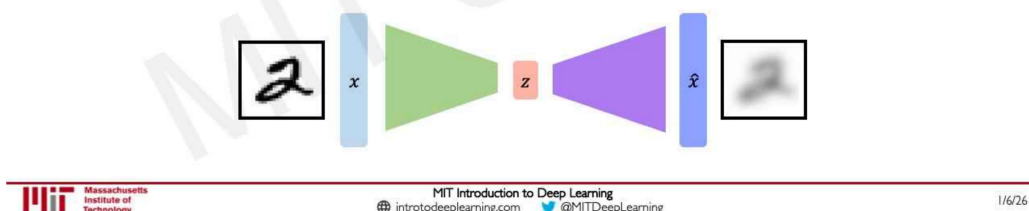


Figure 16: The lecture builds GAN intuition by showing generated points moving toward the real-data region as the discriminator and generator improve in alternation. (MIT 6.S191 Lecture 4 slide 44).

At the idealized optimum, generated samples follow the same distribution as real data, so the discriminator cannot reliably distinguish the two based on distributional differences.

8 GAN Objective Function

The classic minimax GAN objective is

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))].$$

For the **discriminator**, maximizing the first term encourages $D(x)$ to be high for real examples. Maximizing the second term encourages $D(G(z))$ to be low for generated examples, because then $1 - D(G(z))$ is high.

For the **generator**, the adversarial objective pushes generated examples toward outputs that the discriminator classifies as real. The two networks therefore optimize opposing goals.

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

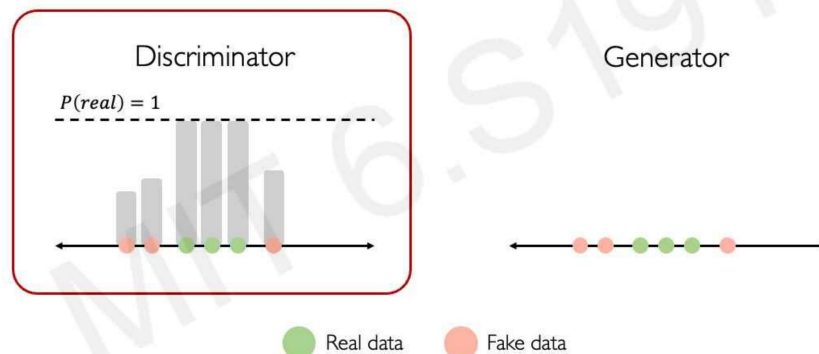


Figure 17: GAN training alternates between improving the discriminator's real/fake classification and improving the generator's ability to fool the discriminator. (MIT 6.S191 Lecture 4 slide 63).

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

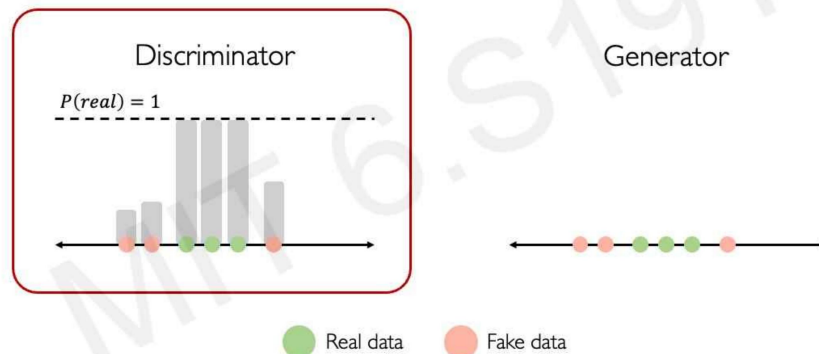


Figure 18: The lecture expresses GAN learning as a minimax objective between G and D . (MIT 6.S191 Lecture 4 slide 64).

8.1 Why GAN training can be difficult

The lecture notes that adversarial training can be unstable. Unlike ordinary supervised learning with one network minimizing one fixed objective, GAN training changes the learning problem for each network as the other network changes. If one network becomes much stronger than the other, useful gradients can become difficult to obtain. This is one reason many GAN variants and training techniques have been developed.

9 Generating New Data with GANs

After training, the discriminator is not normally the component used for generation. We sample a new noise vector z and feed it through G :

$$z \sim p_z(z), \quad x_{\text{new}} = G(z).$$

Different regions of the noise distribution map to different generated examples. If the learned mapping is smooth, moving through the noise/latent space can correspond to semantically smooth changes in generated outputs.

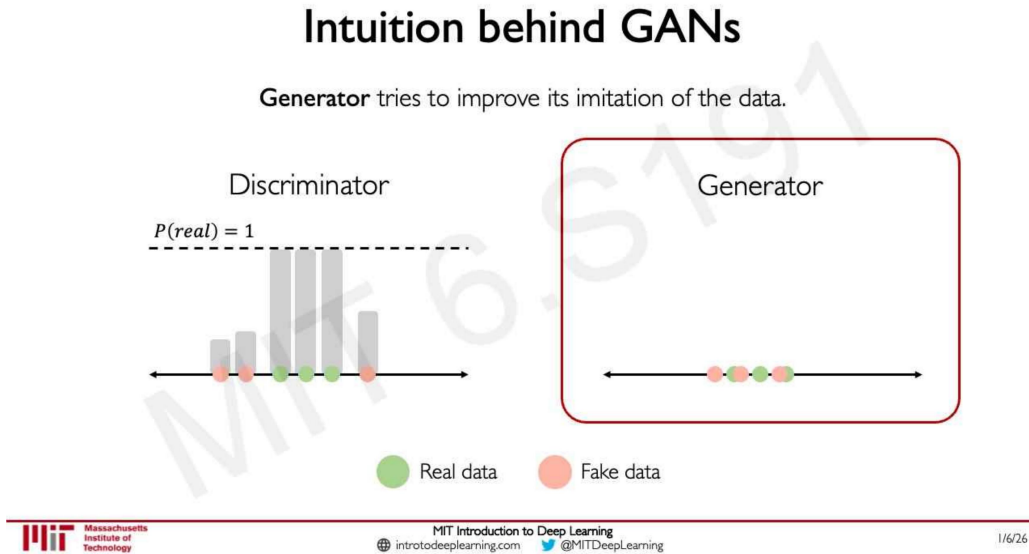


Figure 19: After training, new noise samples are transformed by the generator into new samples from the learned target distribution. (MIT 6.S191 Lecture 4 slide 67).

10 GANs as Distribution Transformers

A standard GAN maps a simple source distribution, often Gaussian noise, into a complex target data distribution. The lecture then broadens this idea: a generator does not necessarily have to transform *noise* into data. It can transform samples from one data domain into another.

This perspective motivates image-to-image and modality-to-modality translation: the source is itself a meaningful data distribution, and the network learns a mapping into a different target distribution.

11 CycleGAN: Unpaired Domain Translation

A **CycleGAN** learns mappings between two domains without requiring every training example in domain A to have a paired counterpart in domain B . It uses two generators,

$$G : A \rightarrow B, \quad F : B \rightarrow A,$$

and discriminators that encourage generated samples to resemble the corresponding target domains.

A central additional idea is **cycle consistency**. If an example a is mapped from A to B and then back again, the result should recover the original example:

$$F(G(a)) \approx a.$$

Similarly,

$$G(F(b)) \approx b.$$

A representative cycle-consistency loss is

$$\mathcal{L}_{\text{cyc}} = \mathbb{E}_{a \sim p_A} \|F(G(a)) - a\|_1 + \mathbb{E}_{b \sim p_B} \|G(F(b)) - b\|_1.$$

Training GANs: loss function

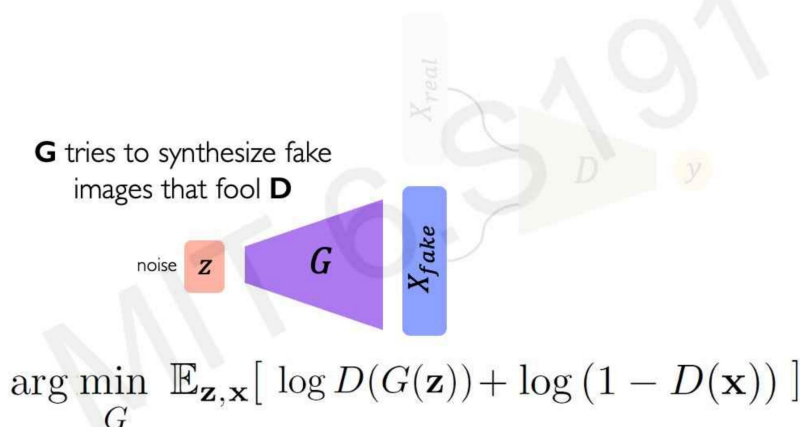


Figure 20: CycleGAN extends adversarial distribution transformation to two domains using forward and reverse mappings constrained by cycle consistency. (MIT 6.S191 Lecture 4 slide 71).

Training GANs: loss function

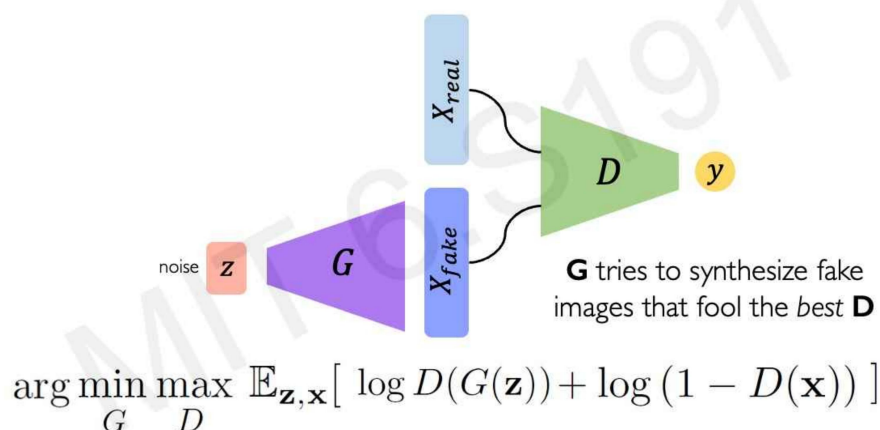


Figure 21: The lecture shows CycleGAN-style transformation beyond ordinary images, including audio-to-audio synthesis. (MIT 6.S191 Lecture 4 slide 72).

The lecture’s key point is that the same distribution-transformation principle can extend across modalities. Audio waveforms, for example, can be transformed between styles directly or through suitable representations.

12 Autoencoders, VAEs, and GANs: Conceptual Comparison

	Autoencoder	VAE	GAN
Primary idea	Compress and reconstruct	Probabilistic latent representation + reconstruction	Adversarial sample generation
Latent representation	Deterministic z	Distribution $q_\phi(z x)$	Noise/latent input to generator
Training signal	Reconstruction loss	Reconstruction + KL regularization	Generator–discriminator adversarial objective
Explicit encoder	Yes	Yes	Not required in standard GAN
Sampling new data	Not naturally reliable	Yes, sample latent space and decode	Yes, sample noise and run generator
Key strength in lecture	Representation learning	Structured, sampleable latent space	Direct generation of new instances

13 Lecture 4 Big Picture

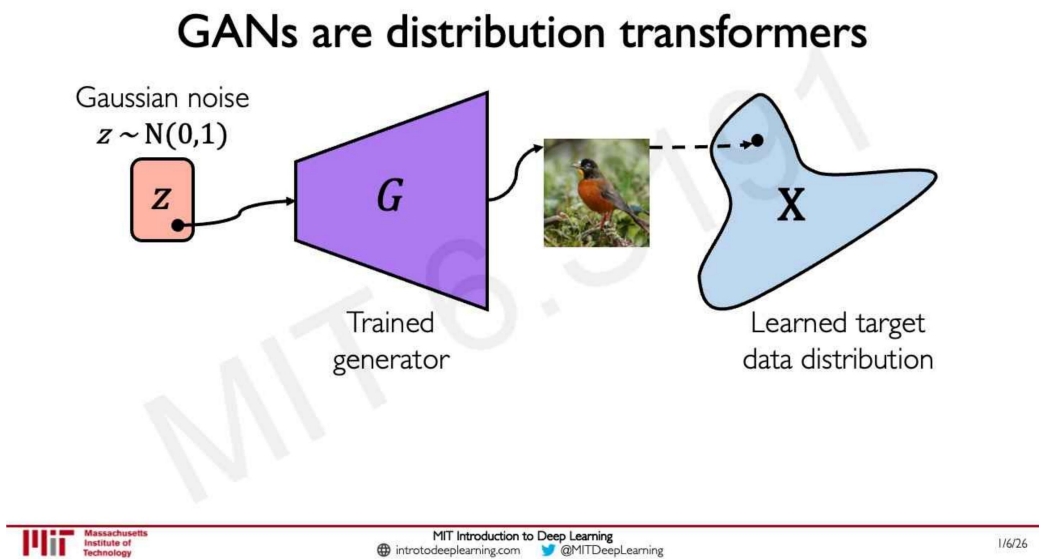


Figure 22: Lecture summary: autoencoding approaches focus on learning latent representations, while GANs focus on learning a generator for new samples. (MIT 6.S191 Lecture 4 slide 75).

The lecture develops generative modeling from one foundational question: **how can a neural network learn the distribution underlying observed data?** Autoencoders first show how reconstruction can provide an unsupervised signal for learning compressed representations. VAEs make that representation

probabilistic, regularize it toward a known prior, and use reparameterization to preserve end-to-end differentiability. GANs approach generation through an adversarial game in which a generator learns to transform a simple distribution into the data distribution. CycleGAN then generalizes distribution transformation from noise-to-data toward domain-to-domain translation.

Exam / interview mental model

Remember the purpose of each component rather than memorizing diagrams:

- **AE:** “Compress enough information to reconstruct x .”
- **VAE:** “Learn a reconstructive latent space that is also probabilistic, smooth, and sampleable.”
- **KL term:** “Keep the inferred latent distribution close to a known prior.”
- **Reparameterization:** “Keep stochastic sampling while allowing gradients to reach μ and σ .”
- **GAN generator:** “Turn simple noise into realistic samples.”
- **GAN discriminator:** “Provide the adversarial learning signal by distinguishing real from generated.”
- **CycleGAN:** “Learn translations between unpaired domains while requiring round-trip consistency.”

14 Quick Review Questions

1. What is the difference between learning $x \rightarrow y$ and learning $p(x)$?
2. Why can reconstruction train an autoencoder without class labels?
3. What does the bottleneck force an autoencoder to learn?
4. Why is an ordinary autoencoder not automatically a good generative model?
5. What do μ and σ represent in a VAE?
6. Why does a VAE need both reconstruction and regularization losses?
7. What do continuity and completeness mean in latent space?
8. Why is $z = \mu + \sigma \odot \epsilon$ useful for backpropagation?
9. What are the competing objectives of G and D in a GAN?
10. After GAN training, which network generates new samples?
11. How does a CycleGAN differ conceptually from a standard noise-to-data GAN?

Source note

These notes are based on the supplied MIT 6.S191 Lecture 4 slides and the accompanying handwritten/typed lecture notes. Slide images are included as study references and are labeled by their original slide number.